HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO BÀI TẬP LỚN IOT VÀ ỨNG DỤNG

Giảng viên hướng dẫn:	**Nguyễn Quốc Uy**
Sinh viên thực hiện:	**Phạm Minh Thiện**
Mã sinh viên:	**B23DCAT285**
Nhóm lớp:	**16**

Hà Nội - 2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU HỆ THỐNG 5
1.1. Giới thiệu và bài toán 5
1.2. Mục tiêu, phạm vi và đối tượng sử dụng 5
1.3. Thành phần và công nghệ lựa chọn 6
CHƯƠNG 2. KIẾN TRÚC TỔNG QUAN CỦA HỆ THỐNG 7
2.1. Sơ đồ tổng quát của hệ thống 7
2.2. Yêu cầu hệ thống 7
2.2.1. Yêu cầu chức năng 8
2.2.2. Yêu cầu phi chức năng 8
2.2.3. Nguyên tắc thiết kế UX/UI 8
2.3. Thiết kế các lớp kiến trúc 9
2.4. Thiết kế phần cứng 9
2.5. Thiết kế phần mềm 11
2.6. Thiết kế cơ sở dữ liệu 11
2.6.1. Sơ đồ quan hệ ERD 12
2.6.2. Bảng users 12
2.6.3. Bảng sensors 13
2.6.4. Bảng devices 13
2.6.5. Bảng sensor_data 14
2.6.6. Bảng action_history 14
2.6.7. Quy tắc dữ liệu quan trọng 14
2.7. Thiết kế luồng dữ liệu và giao tiếp 15
2.8. MQTT topic sử dụng 16

CHƯƠNG 3. THIẾT KẾ USE CASE 17
3.1. Use case: Dashboard 18
a. Đặc tả use case 18
b. Sequence diagram 19
c. Mô tả API 19
d. Dữ liệu đọc/ghi tại CSDL 19
e. Mapping dữ liệu API - Database 19
3.2. Use case: Xem dữ liệu cảm biến thời gian thực 20
a. Đặc tả use case 20
b. Sequence diagram 21
c. Mô tả API 21
d. Dữ liệu đọc/ghi tại CSDL 21
e. Mapping dữ liệu API - Database 21
3.3. Use case: Xem biểu đồ dữ liệu cảm biến 22
a. Đặc tả use case 22
b. Sequence diagram 23
c. Mô tả API 23
d. Dữ liệu đọc/ghi tại CSDL 23
e. Mapping dữ liệu API - Database 23

3.4. Use case: Điều khiển bật/tắt thiết bị....24
a. Đặc tả use case....24
b. Sequence diagram....25
c. Mô tả API....25
d. Dữ liệu đọc/ghi tại CSDL....25
e. Mapping dữ liệu API - Database....26
3.5. Use case: Xem lịch sử giá trị cảm biến....27
a. Đặc tả use case....27
b. Sequence diagram....28
c. Mô tả API....28
d. Dữ liệu đọc/ghi tại CSDL....28
e. Mapping dữ liệu API - Database....28
3.6. Use case: Xem lịch sử điều khiển thiết bị....29
a. Đặc tả use case....29
b. Sequence diagram....30
c. Mô tả API....30
d. Dữ liệu đọc/ghi tại CSDL....30
e. Mapping dữ liệu API - Database....30

CHƯƠNG 1. GIỚI THIỆU HỆ THỐNG

1.1. Giới thiệu và bài toán

Internet of Things (IoT) cho phép các thiết bị vật lý thu thập dữ liệu, kết nối mạng và phối hợp với phần mềm để tạo thành một hệ thống giám sát - điều khiển thống nhất. Trong phạm vi bài tập lớn, hệ thống được xây dựng để theo dõi nhiệt độ, độ ẩm và cường độ ánh sáng; đồng thời hỗ trợ bật/tắt các thiết bị điện từ xa thông qua giao diện Web.

Bài toán đặt ra là dữ liệu môi trường thay đổi liên tục nhưng người dùng không thể quan sát trực tiếp tại mọi thời điểm. Nếu chỉ ghi nhận thủ công, dữ liệu dễ bị gián đoạn, khó tra cứu và không có căn cứ để đánh giá trạng thái theo thời gian. Tương tự, thao tác bật/tắt thiết bị tại chỗ không đáp ứng yêu cầu điều khiển từ xa và không lưu được lịch sử ai đã thực hiện, thiết bị nào đã thay đổi và kết quả có thành công hay không.

Hệ thống vì vậy cần một nút IoT dùng ESP8266 để đọc cảm biến và nhận lệnh, một MQTT Broker làm trung gian truyền thông, Backend để kiểm tra nghiệp vụ và lưu dữ liệu, cơ sở dữ liệu quan hệ để bảo đảm tính nhất quán, cùng Frontend trực quan cho người dùng. Thiết kế phải đồng thời giải quyết các yêu cầu về thời gian thực, truy vết lịch sử, xác thực người dùng, khả năng mở rộng và xử lý lỗi kết nối.

1.2. Mục tiêu, phạm vi và đối tượng sử dụng

Mục tiêu của hệ thống gồm các nội dung chính sau:

- Thu thập nhiệt độ, độ ẩm và ánh sáng theo chu kỳ từ các cảm biến gắn với ESP8266.
- Truyền dữ liệu và lệnh điều khiển qua MQTT; Backend không phụ thuộc vào kết nối trực tiếp với từng thiết bị.
- Hiển thị dữ liệu mới nhất, biểu đồ biến thiên và lịch sử đo trên giao diện Web.
- Cho phép người dùng đã đăng nhập bật/tắt thiết bị; kết quả chỉ được xác nhận sau khi ESP8266 phản hồi.
- Lưu đầy đủ lịch sử điều khiển, hỗ trợ tìm kiếm, lọc, sắp xếp và phân trang.
- Bảo đảm API có xác thực, kiểm tra đầu vào, xử lý timeout và không lưu mật khẩu dạng văn bản thuần.

Phạm vi thực hiện:

Hệ thống tập trung vào một mô hình giám sát trong một khu vực với ba nhóm dữ liệu môi trường và ba kênh relay mô phỏng Quạt, Đèn và Điều hòa. Báo cáo mô tả đầy đủ kiến trúc, phần cứng, phần mềm, cơ sở dữ liệu, MQTT topic, API và sáu use case trọng tâm. Các nội dung như thanh toán, điều khiển công suất lớn, triển khai đa vùng hoặc ứng dụng di động không thuộc phạm vi bài tập.

Đối tượng sử dụng:

- Người dùng: đăng nhập, quan sát dữ liệu, xem biểu đồ/lịch sử và điều khiển thiết bị.
- Quản trị viên: quản lý cấu hình, theo dõi lỗi, kiểm tra nhật ký và tài khoản (hướng mở rộng).
- ESP8266: actor hệ thống gửi dữ liệu cảm biến, nhận lệnh và phản hồi trạng thái.

1.3. Thành phần và công nghệ lựa chọn

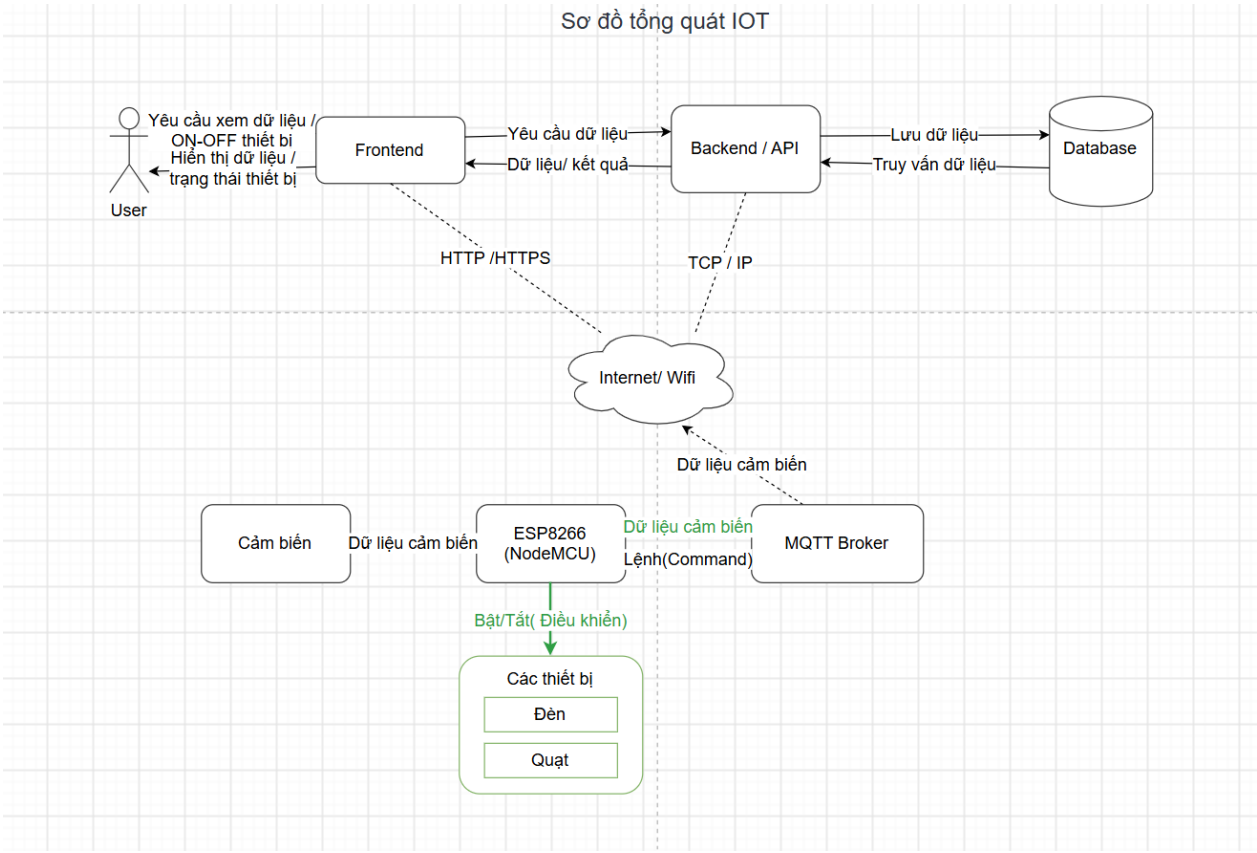
Các thành phần được lựa chọn theo tiêu chí dễ triển khai trong môi trường học tập, có cộng đồng hỗ trợ lớn, giao tiếp chuẩn và đủ khả năng mở rộng từ mô hình thử nghiệm sang hệ thống nhiều thiết bị.

Nhóm	Thành phần/công nghệ	Vai trò và lý do lựa chọn
Vi điều khiển	ESP8266 NodeMCU	Tích hợp WiFi, đủ GPIO, hỗ trợ Arduino C++ và thư viện MQTT.
Cảm biến	DHT11; BH1750	Đo nhiệt độ/độ ẩm và ánh sáng; giao tiếp GPIO/I²C đơn giản.
Chấp hành	Relay 3 kênh	Đóng/cắt tải mô phỏng Quạt, Đèn và Điều hòa.
Giao thức	MQTT 3.1.1	Nhẹ, mô hình publish/subscribe, phù hợp thiết bị tài nguyên hạn chế.
Broker	Eclipse Mosquitto	Mã nguồn mở, cài đặt đơn giản, hỗ trợ QoS và xác thực.
Backend	Node.js + Express + MQTT.js	Xử lý REST API và sự kiện MQTT trong cùng mô hình bất đồng bộ.
Frontend	ReactJS + Axios + Chart.js	Tổ chức giao diện theo component, gọi API và hiển thị biểu đồ.
Cơ sở dữ liệu	MySQL	Ràng buộc quan hệ rõ ràng, hỗ trợ chỉ mục, giao dịch và truy vấn tổng hợp.
Xác thực	JWT + bcrypt	API không duy trì session phía server; mật khẩu được băm trước khi lưu.

CHƯƠNG 2. KIẾN TRÚC TỔNG QUAN CỦA HỆ THỐNG

Chương này trình bày sơ đồ tổng quát, yêu cầu chức năng và phi chức năng, nguyên tắc UX/UI, các lớp kiến trúc và những thành phần triển khai. Mỗi quyết định đều bảo đảm dữ liệu đi qua Backend trước khi được lưu hoặc hiển thị; MQTT được dùng cho giao tiếp với ESP8266, còn REST API được dùng giữa Frontend và Backend.

2.1. Sơ đồ tổng quát của hệ thống



Hình 2.1. Sơ đồ tổng quát của hệ thống IoT

Người dùng làm việc trên Frontend bằng trình duyệt. Frontend chỉ gọi REST API qua HTTP/HTTPS và không truy cập trực tiếp MySQL hoặc MQTT Broker. Backend chịu trách nhiệm xác thực, kiểm tra dữ liệu, thực hiện nghiệp vụ, truy vấn cơ sở dữ liệu và quản lý MQTT client. ESP8266 trao đổi telemetry, command và status với Broker; nhờ đó các thành phần được tách rời và có thể thay thế độc lập.

- Kênh Web: User ↔ Frontend ↔ REST Backend, dữ liệu trao đổi ở dạng JSON.
- Kênh thiết bị: Backend ↔ MQTT Broker ↔ ESP8266 theo mô hình publish/subscribe.
- Kênh dữ liệu: Backend ↔ MySQL; chỉ Backend có quyền đọc/ghi cơ sở dữ liệu.

2.2. Yêu cầu hệ thống

Yêu cầu được xác định theo phạm vi bài tập lớn và dùng làm căn cứ đối chiếu khi thiết kế use case, API, cơ sở dữ liệu và giao diện. Các chỉ tiêu thời gian dưới đây là mục tiêu thiết kế, được kiểm tra lại khi triển khai thực tế.

2.2.1. Yêu cầu chức năng

Mã	Yêu cầu	Tiêu chí đáp ứng
FR01	Đăng nhập và xác thực người dùng.	Đúng tài khoản trả JWT; sai thông tin trả 401.
FR02	Thu thập nhiệt độ, độ ẩm và ánh sáng từ ESP8266.	Telemetry hợp lệ được ghi vào sensor_data.
FR03	Xem dữ liệu cảm biến mới nhất trên Dashboard.	Hiển thị đủ tên cảm biến, ba giá trị, thời gian và trạng thái.
FR04	Xem biểu đồ dữ liệu cảm biến theo khoảng thời gian.	Ba series dùng cùng mốc thời gian và hỗ trợ bộ lọc.
FR05	Điều khiển bật/tắt thiết bị từ xa.	Lệnh có actionCode, nhận ACK và lưu action_history.
FR06	Tra cứu lịch sử giá trị cảm biến.	Có tìm kiếm, lọc, sắp xếp và phân trang.
FR07	Tra cứu lịch sử hoạt động thiết bị.	Hiển thị thiết bị, action ON/OFF, thời gian và kết quả.

2.2.2. Yêu cầu phi chức năng

Mã	Nhóm	Yêu cầu/tiêu chí thiết kế
NFR01	Hiệu năng	API đọc dữ liệu thông thường hướng tới phản hồi trong 2 giây ở mô hình học tập.
NFR02	Thời gian thực	Dashboard cập nhật bản ghi mới trong chu kỳ tối đa 5 giây sau khi Backend nhận telemetry.
NFR03	Tin cậy	Lệnh điều khiển chỉ báo thành công sau ACK; quá thời gian chờ phải ghi FAILED.
NFR04	Bảo mật	JWT cho API; mật khẩu băm bcrypt; validate đầu vào; MQTT dùng tài khoản/ACL và TLS khi triển khai thật.
NFR05	Toàn vẹn	Khóa ngoại, UNIQUE, CHECK, transaction và actionCode bảo đảm dữ liệu nhất quán.
NFR06	Khả dụng	Giao diện dễ đọc, có cảnh báo trạng thái, bộ lọc rõ ràng và dùng được trên màn hình phổ biến.

2.2.3. Nguyên tắc thiết kế UX/UI

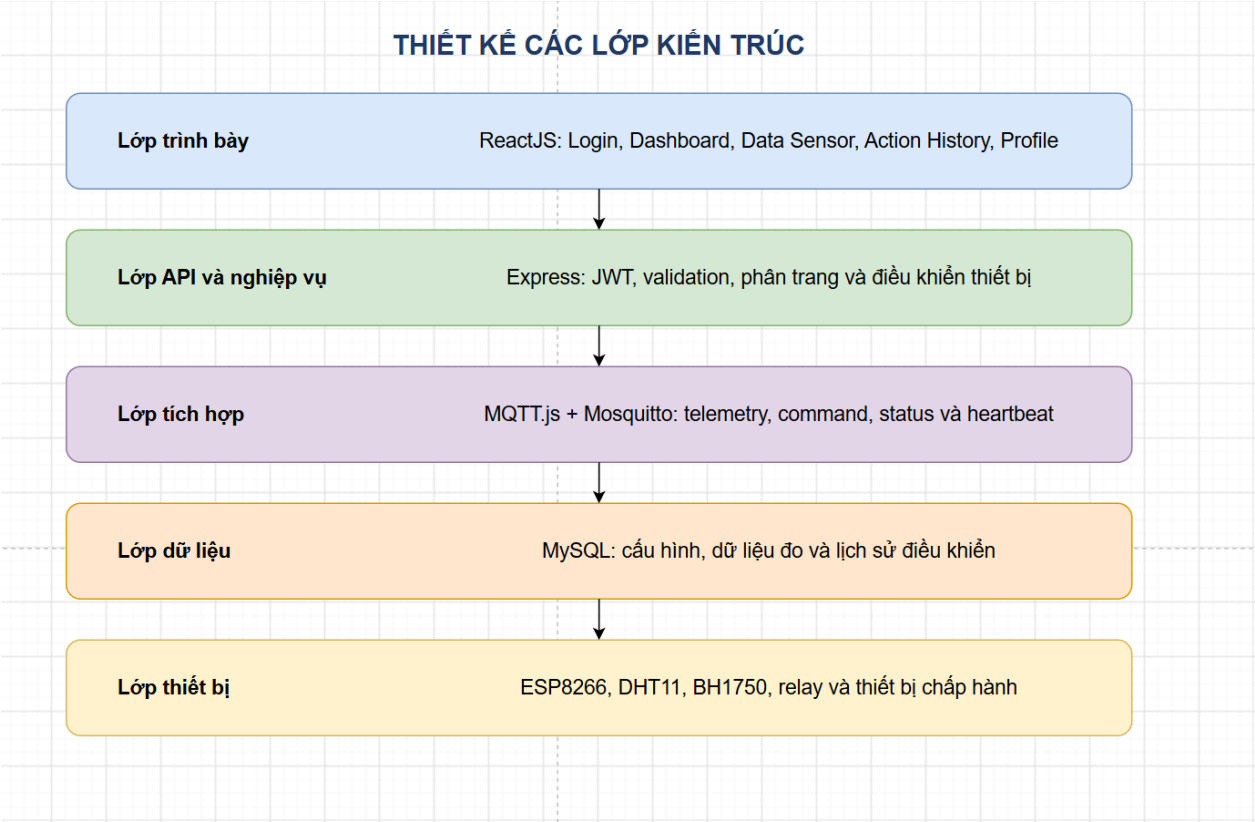
Thông tin phục vụ quyết định của người dùng được đặt trước; trường kỹ thuật hoặc ít dùng được đặt cuối. Cách sắp xếp này giảm thời gian quét bảng và vẫn giữ đủ dữ liệu để đối chiếu khi cần.

Giao diện	Thứ tự trường hiển thị	Lý do
Data Sensor	Tên cảm biến → Nhiệt độ → Độ ẩm → Ánh sáng → Thời gian → Trạng thái → ID	Tên và giá trị đo là thông tin chính; data_id tự tăng là trường kỹ thuật nên đặt cuối.

Giao diện	Thứ tự trường hiển thị	Lý do
Action History	Tên thiết bị → Hành động → Thời gian → Kết quả → Người thao tác → Khu vực → ID	Hành động và kết quả cần được thấy trước; history_id chỉ dùng khi tra cứu chi tiết.
Dashboard	Giá trị mới nhất → Cảnh báo → Biểu đồ → Điều khiển	Ưu tiên quan sát tình trạng hiện tại trước thao tác điều khiển.

- ID do cơ sở dữ liệu tự sinh, Frontend không cho người dùng nhập hoặc sửa.
- Bộ lọc chỉ hiển thị các trường có giá trị tra cứu; trạng thái dùng màu nhưng luôn kèm chữ.
- Các cột kỹ thuật có thể ẩn trên màn hình hẹp và xuất hiện đầy đủ trong trang chi tiết/xuất dữ liệu.

2.3. Thiết kế các lớp kiến trúc

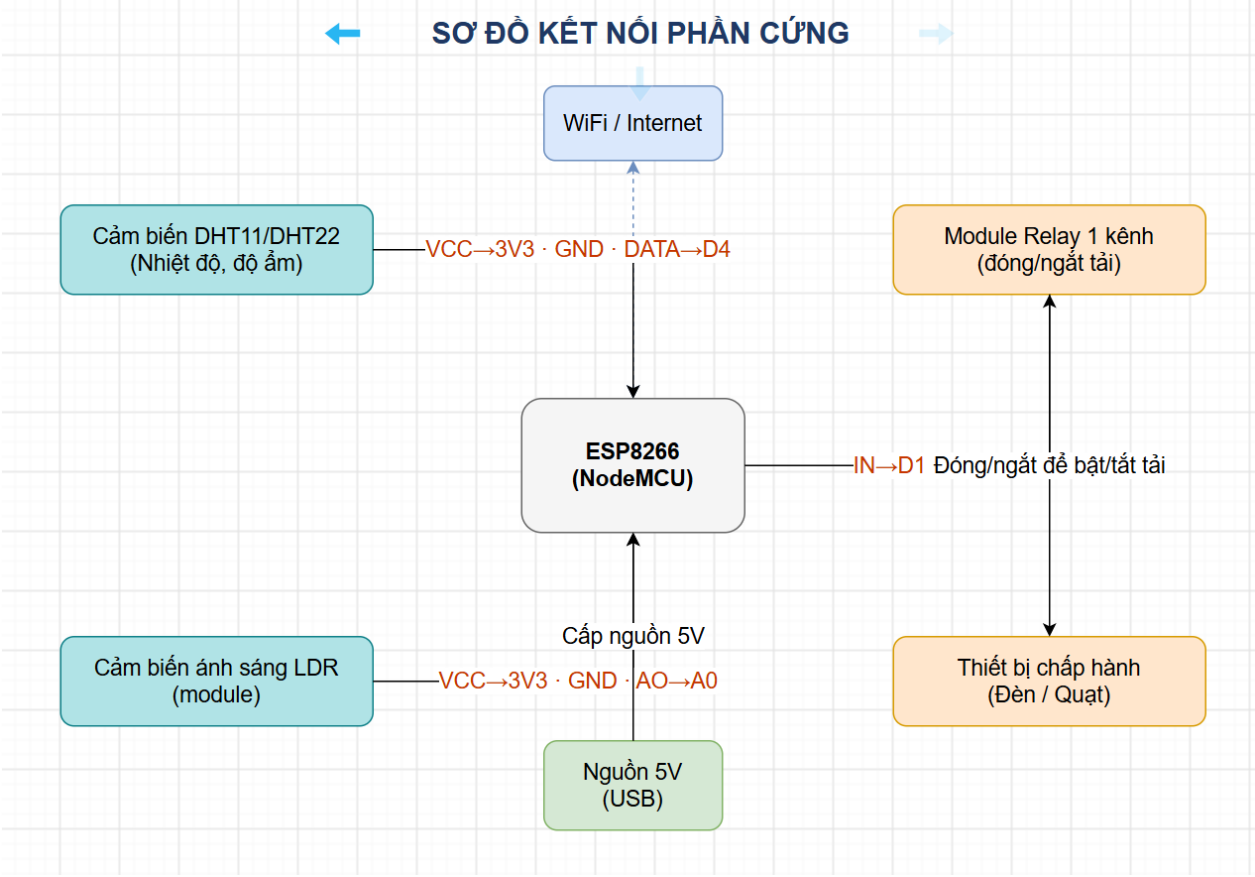


Hình 2.2. Các lớp kiến trúc và trách nhiệm chính

Lớp	Trách nhiệm	Đầu vào/đầu ra
Trình bày	Hiển thị màn hình, kiểm tra định dạng cơ bản, quản lý token.	Thao tác User ↔ REST/JSON
API và nghiệp vụ	Xác thực, phân quyền, kiểm tra quy tắc, điều phối dịch vụ.	HTTP request ↔ JSON response
Tích hợp	Subscribe telemetry/status; publish command; xử lý reconnect.	MQTT message
Dữ liệu	Lưu, truy vấn, phân trang, tổng hợp và bảo đảm toàn vẹn.	SQL ↔ bản ghi
Thiết bị	Đọc cảm biến, điều khiển relay, gửi heartbeat và ACK.	GPIO/I²C ↔ MQTT

2.4. Thiết kế phần cứng

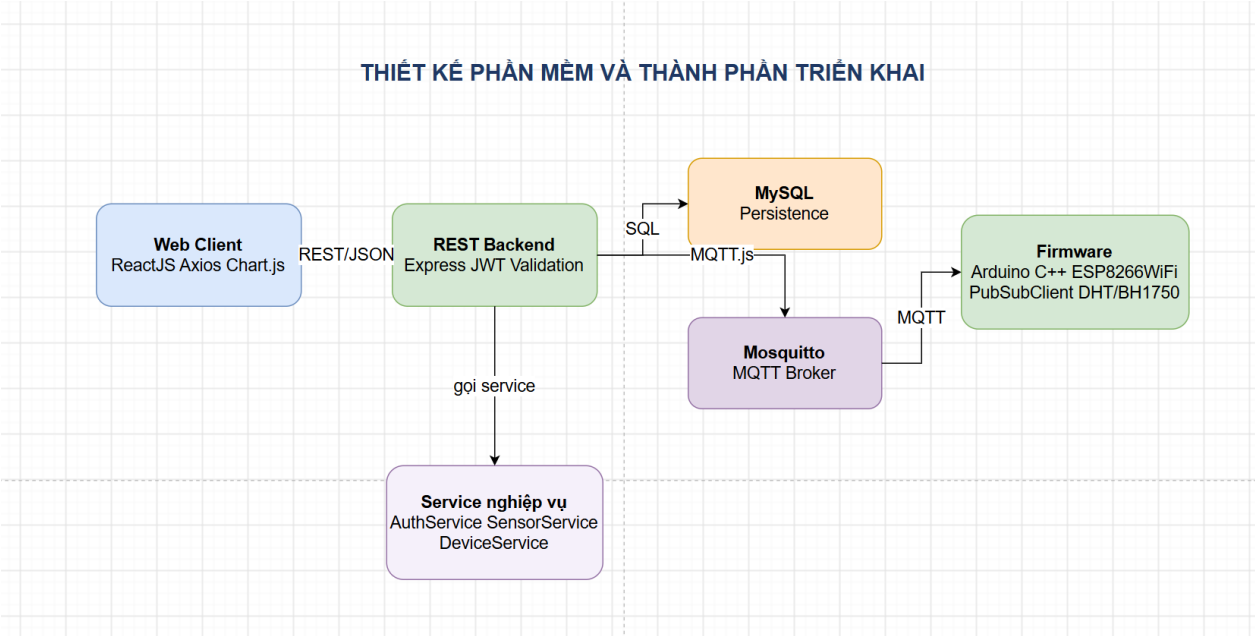
ESP8266 NodeMCU là bộ điều khiển trung tâm. DHT11 cung cấp nhiệt độ và độ ẩm qua một chân dữ liệu; BH1750 đo cường độ ánh sáng qua I²C. Ba chân số điều khiển module relay, mỗi kênh tương ứng một thiết bị chấp hành. Trong mô hình học tập, tải điện áp thấp hoặc đèn LED được ưu tiên để bảo đảm an toàn.



Hình 2.3. Sơ đồ khối phần cứng

Thành phần	Chân NodeMCU	Giao tiếp	Ghi chú
DHT11	D4 / GPIO2	Digital	Điện trở kéo lên 10 kΩ nếu module không tích hợp.
BH1750	D2 / GPIO4 (SDA); D1 / GPIO5 (SCL)	I²C	Địa chỉ mặc định 0x23.
Relay Quạt	D5 / GPIO14	Digital output	Mức kích tùy module relay.
Relay Đèn	D6 / GPIO12	Digital output	Khởi tạo về OFF khi boot.
Relay Điều hòa	D7 / GPIO13	Digital output	Dùng tải mô phỏng trong bài tập.
Nguồn	VIN/5 V; 3V3	Power	Nối chung GND; không cấp tải công suất từ chân GPIO.

2.5. Thiết kế phần mềm



Hình 2.4. Các thành phần phần mềm và giao thức kết nối

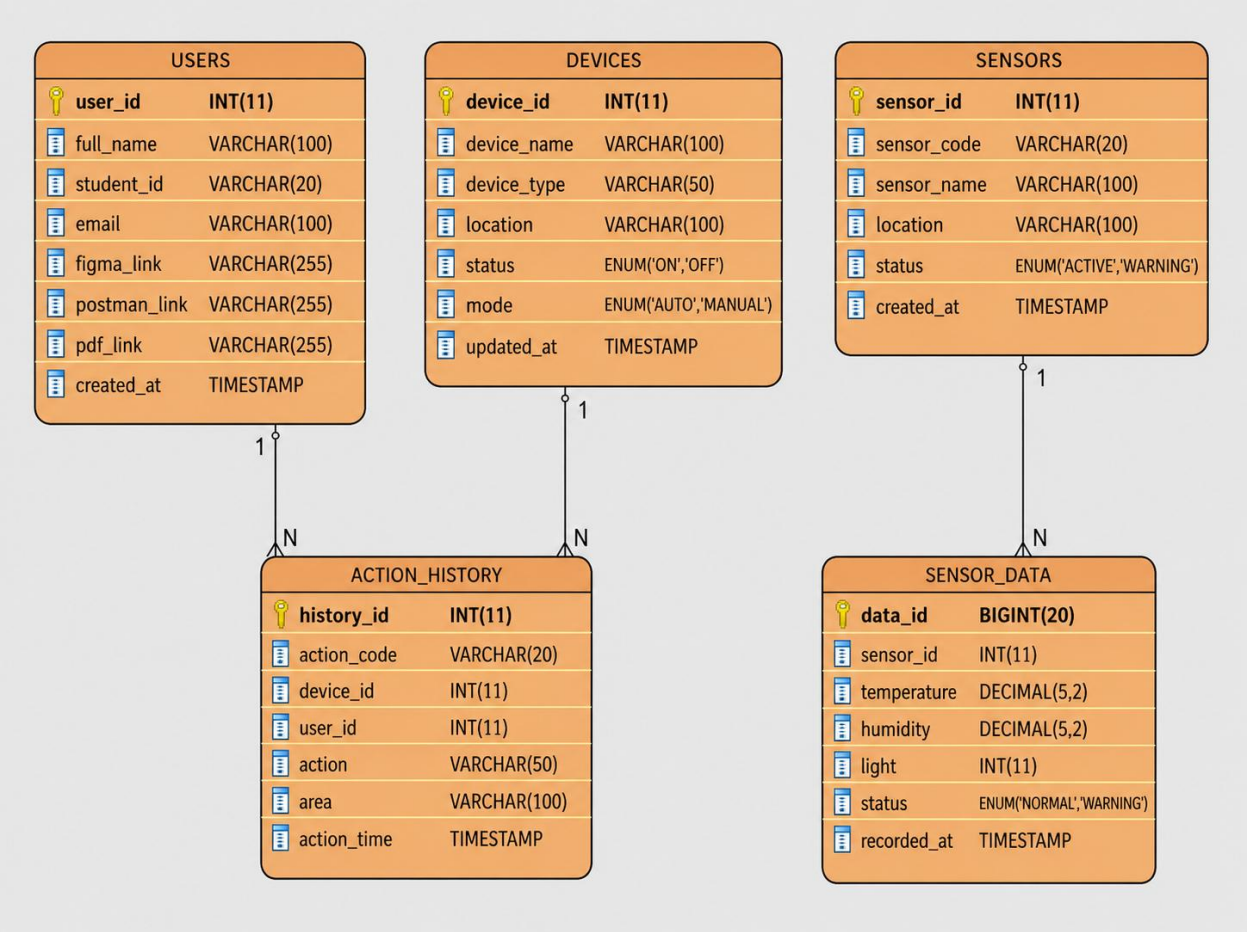
Backend được tổ chức thành controller, service, repository và MQTT handler. Controller tiếp nhận HTTP request; service thực hiện nghiệp vụ; repository thao tác MySQL; MQTT handler quản lý subscribe/publish và liên kết ACK với yêu cầu điều khiển. Cách tách này giúp kiểm thử từng lớp, tránh viết SQL hoặc publish MQTT trực tiếp trong route.

Thành phần	Module chính	Chức năng
Frontend	Auth, Dashboard, m , ActionHistory, Profile	Giao diện, biểu đồ, bảng dữ liệu, bộ lọc và trạng thái thiết bị.
Backend	AuthController, SensorService, DeviceService, HistoryService	REST API, JWT, validation, nghiệp vụ và xử lý lỗi.
MQTT handler	TelemetryConsumer, CommandPublisher, StatusConsumer	Nhận dữ liệu, gửi lệnh, đối chiếu actionCode và timeout.
Database	Repository/DAO + transaction	CRUD, phân trang, tổng hợp biểu đồ và cập nhật trạng thái.
Firmware	SensorTask, MqttTask, RelayController	Đọc cảm biến, reconnect WiFi/MQTT, điều khiển relay và phản hồi.

2.6. Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu sử dụng năm bảng cốt lõi. Thiết kế tách dữ liệu cấu hình (users, sensors, devices) khỏi dữ liệu phát sinh theo thời gian (sensor_data, action_history), nhờ đó tránh lặp thông tin và hỗ trợ truy vấn lịch sử hiệu quả.

2.6.1. Sơ đồ quan hệ ERD



Hình 2.5. Sơ đồ quan hệ ERD đã bổ sung dữ liệu xác thực

Quan hệ chính gồm users 1-N action_history, devices 1-N action_history và sensors 1-N sensor_data. Bảng action_history lưu đồng thời user_id và device_id để truy vết người thao tác và thiết bị bị tác động. Bảng users bổ sung username và password_hash để triển khai use case đăng nhập, khắc phục phần còn thiếu trong thiết kế cũ.

2.6.2. Bảng users

Bảng users lưu tài khoản đăng nhập và thông tin hồ sơ của người sử dụng.

Trường	Kiểu dữ liệu	Khóa/Ràng buộc	NULL	Ý nghĩa
user_id	INT	PK, AUTO_INCREMENT	Không	Định danh người dùng.
username	VARCHAR(50)	UNIQUE	Không	Tên đăng nhập duy nhất.
password_hash	VARCHAR(255)	-	Không	Mật khẩu đã băm bằng bcrypt.
full_name	VARCHAR(100)	-	Không	Họ và tên.
student_id	VARCHAR(20)	UNIQUE	Có	Mã sinh viên.
email	VARCHAR(100)	UNIQUE	Có	Email liên hệ.

Trường	Kiểu dữ liệu	Khóa/Ràng buộc	NULL	Ý nghĩa
role	ENUM	USER/ADMIN	Không	Vai trò để phân quyền.
figma_link	VARCHAR(255)	-	Có	Liên kết Figma.
postman_link	VARCHAR(255)	-	Có	Liên kết Postman.
pdf_link	VARCHAR(255)	-	Có	Liên kết báo cáo.
created_at	TIMESTAMP	DEFAULT CURRENT_ TIMESTAMP	Không	Thời điểm tạo.
updated_at	TIMESTAMP	ON UPDATE	Không	Thời điểm cập nhật.

2.6.3. Bảng sensors

Bảng sensors quản lý nút/cụm cảm biến gửi dữ liệu môi trường.

Trường	Kiểu dữ liệu	Khóa/Ràng buộc	NULL	Ý nghĩa
sensor_id	INT	PK, AUTO_INCREMENT	Không	Định danh cảm biến.
sensor_code	VARCHAR(30)	UNIQUE	Không	Mã dùng trong MQTT topic.
sensor_name	VARCHAR(100)	-	Không	Tên hiển thị.
sensor_type	VARCHAR(50)	-	Không	Loại cụm cảm biến.
unit	VARCHAR(20)	-	Có	Đơn vị hoặc mô tả nhóm giá trị.
location	VARCHAR(100)	-	Có	Vị trí lắp đặt.
status	ENUM	ACTIVE/WARNING	Không	Trạng thái hoạt động.
created_at	TIMESTAMP	DEFAULT CURRENT_ TIMESTAMP	Không	Thời điểm tạo.

2.6.4. Bảng devices

Bảng devices lưu thông tin và trạng thái hiện tại của thiết bị chấp hành.

Trường	Kiểu dữ liệu	Khóa/Ràng buộc	NULL	Ý nghĩa
device_id	INT	PK, AUTO_INCREMENT	Không	Định danh thiết bị.
device_code	VARCHAR(30)	UNIQUE	Không	Mã dùng trong topic command/status.
device_name	VARCHAR(100)	-	Không	Tên hiển thị.
device_type	VARCHAR(50)	-	Không	Quạt, đèn hoặc thiết bị khác.
location	VARCHAR(100)	-	Có	Khu vực lắp đặt.
status	ENUM	ON/OFF	Không	Trạng thái được ESP8266 xác nhận.
mode	ENUM	AUTO/MANUAL	Không	Chế độ điều khiển.
last_seen	TIMESTAMP	-	Có	Lần cuối nhận status/heartbeat.

Trường	Kiểu dữ liệu	Khóa/Ràng buộc	NULL	Ý nghĩa
updated_at	TIMESTAMP	ON UPDATE	Không	Thời điểm cập nhật.

2.6.5. Bảng sensor_data

Bảng sensor_data là bảng dữ liệu thời gian, mỗi dòng tương ứng một lần đo. Trường data_id được tự động tăng tại CSDL; Frontend không gửi giá trị này khi ghi dữ liệu.

Trường	Kiểu dữ liệu	Khóa/Ràng buộc	NULL	Ý nghĩa
data_id	BIGINT	PK, AUTO_INCREMENT	Không	Định danh bản ghi.
sensor_id	INT	FK → sensors	Không	Cảm biến gửi dữ liệu.
temperature	DECIMAL(5,2)	CHECK -40..125	Không	Nhiệt độ (°C).
humidity	DECIMAL(5,2)	CHECK 0..100	Không	Độ ẩm (%).
light	INT	CHECK ≥ 0	Không	Cường độ ánh sáng (lux).
status	ENUM	NORMAL/WARNING	Không	Kết quả so ngưỡng.
recorded_at	TIMESTAMP	INDEX	Không	Thời điểm đo.

2.6.6. Bảng action_history

Bảng action_history lưu vòng đời của từng lệnh điều khiển từ lúc yêu cầu đến khi nhận ACK hoặc timeout. history_id được tự động tăng; action lưu trực tiếp giá trị nghiệp vụ như ON hoặc OFF.

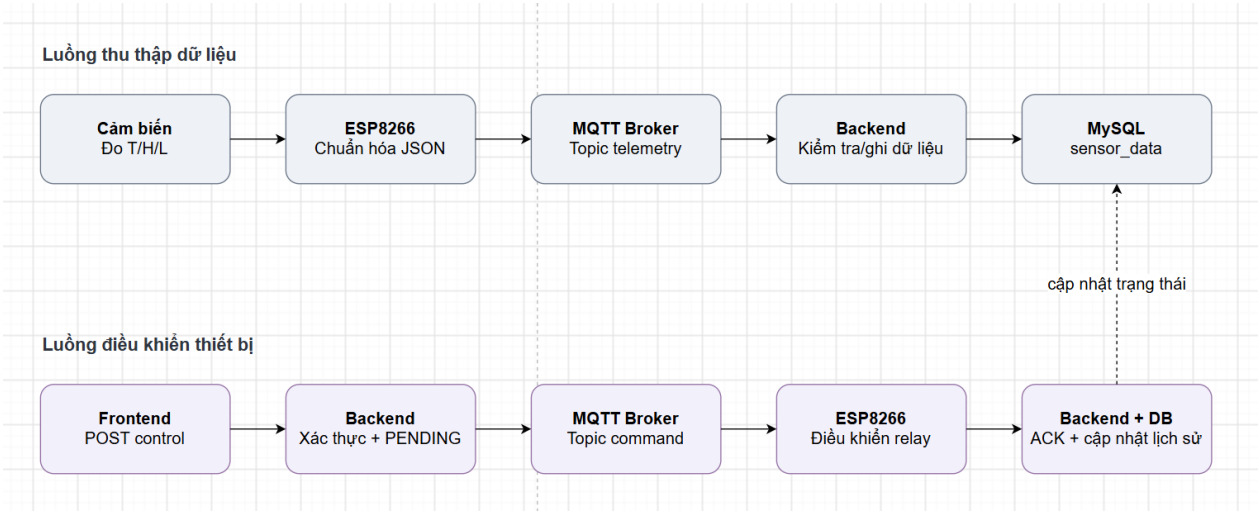
Trường	Kiểu dữ liệu	Khóa/Ràng buộc	NULL	Ý nghĩa
history_id	BIGINT	PK, AUTO_INCREMENT	Không	Định danh bản ghi.
action_code	VARCHAR(36)	UNIQUE	Không	Mã đối chiếu request và MQTT ACK.
device_id	INT	FK → devices	Không	Thiết bị được điều khiển.
user_id	INT	FK → users	Không	Người thực hiện.
action	VARCHAR(20)	ON/OFF/AUTO	Không	Hành động yêu cầu.
requested_state	VARCHAR(20)	-	Không	Trạng thái mong muốn.
result	ENUM	PENDING/SUCCESS/FAILED	Không	Kết quả xử lý.
area	VARCHAR(100)	-	Có	Khu vực tại thời điểm điều khiển.
action_time	TIMESTAMP	INDEX	Không	Thời điểm tạo yêu cầu.

2.6.7. Quy tắc dữ liệu quan trọng

- username, sensor_code, device_code và action_code phải duy nhất; Backend xử lý xung đột với mã lỗi 409.

- password_hash chỉ lưu chuỗi bcrypt; API không bao giờ trả password_hash về Frontend.
- sensor_data chỉ được ghi khi sensor_code tồn tại và ba giá trị nằm trong miền hợp lệ.
- devices.status chỉ cập nhật sau ACK của ESP8266; gửi command không đồng nghĩa thiết bị đã đổi trạng thái.
- Mỗi lệnh điều khiển tạo action_history = PENDING, sau đó chuyển SUCCESS hoặc FAILED, không xóa bản ghi.
- Tạo chỉ mục trên recorded_at, action_time và các khóa ngoại để tối ưu lọc theo thời gian và phân trang.
- Câu SQL sử dụng tham số bind; tên cột sort phải đi qua whitelist, không ghép trực tiếp dữ liệu người dùng.

2.7. Thiết kế luồng dữ liệu và giao tiếp



Hình 2.6. Hai luồng dữ liệu trọng tâm của hệ thống

Luồng thu thập dữ liệu:

1. ESP8266 đọc cảm biến theo chu kỳ, bổ sung sensorCode và timestamp rồi tạo payload JSON.
2. Thiết bị publish lên topic telemetry với QoS 1; Broker chuyển message cho Backend đã subscribe.
3. Backend parse JSON, kiểm tra kiểu/miền giá trị, xác định NORMAL hoặc WARNING và ghi sensor_data.
4. Frontend gọi API latest/chart/history để lấy dữ liệu đã được Backend chuẩn hóa.

Luồng điều khiển thiết bị:

1. Frontend gửi deviceId và action cùng JWT đến Backend.
2. Backend kiểm tra quyền, tạo actionCode, ghi action_history = PENDING rồi publish command.
3. ESP8266 nhận lệnh, điều khiển relay và publish status chứa cùng actionCode.
4. Backend đối chiếu ACK, cập nhật devices và action_history; nếu quá thời gian chờ thì ghi FAILED.

2.8. MQTT topic sử dụng

Topic	Publisher	Subscriber	QoS	Mục đích
iot/sensors/{sensorCode}/data	ESP8266	Backend	1	Gửi nhiệt độ, độ ẩm, ánh sáng.
iot/devices/{deviceCode}/command	Backend	ESP8266	1	Gửi ON/OFF/AUTO kèm actionCode.
iot/devices/{deviceCode}/status	ESP8266	Backend	1	ACK trạng thái thực tế và kết quả.
iot/nodes/{nodeId}/heartbeat	ESP8266	Backend	0	Thông báo thiết bị đang online.
iot/nodes/{nodeId}/config	Backend	ESP8266	1	Cập nhật chu kỳ đo/ngưỡng (mở rộng).

Ví dụ payload telemetry và command:

```
// Telemetry
{
  "sensorCode": "SENSOR_01",
  "temperature": 28.9,
  "humidity": 61.5,
  "light": 468.7,
  "timestamp": "2026-08-24T13:20:15Z"
}

// Command
{
  "actionCode": "9a8b...",
  "deviceCode": "FAN_01",
  "action": "ON",
  "mode": "MANUAL",
  "requestedAt": "2026-08-24T13:21:00Z"
}
```

Quy ước vận hành MQTT:

- Telemetry, command và status dùng QoS 1; Backend xử lý khả năng nhận message trùng dựa trên actionCode hoặc timestamp.
- Không đặt retained cho command để tránh ESP8266 nhận lại lệnh cũ sau khi reconnect; status có thể retained nếu cần đồng bộ trạng thái gần nhất.
- Heartbeat kết hợp Last Will and Testament để đánh dấu node offline khi mất kết nối bất thường.
- Môi trường triển khai thật cần tài khoản riêng cho từng client, Access Control List theo topic và TLS.

CHƯƠNG 3. THIẾT KẾ USE CASE

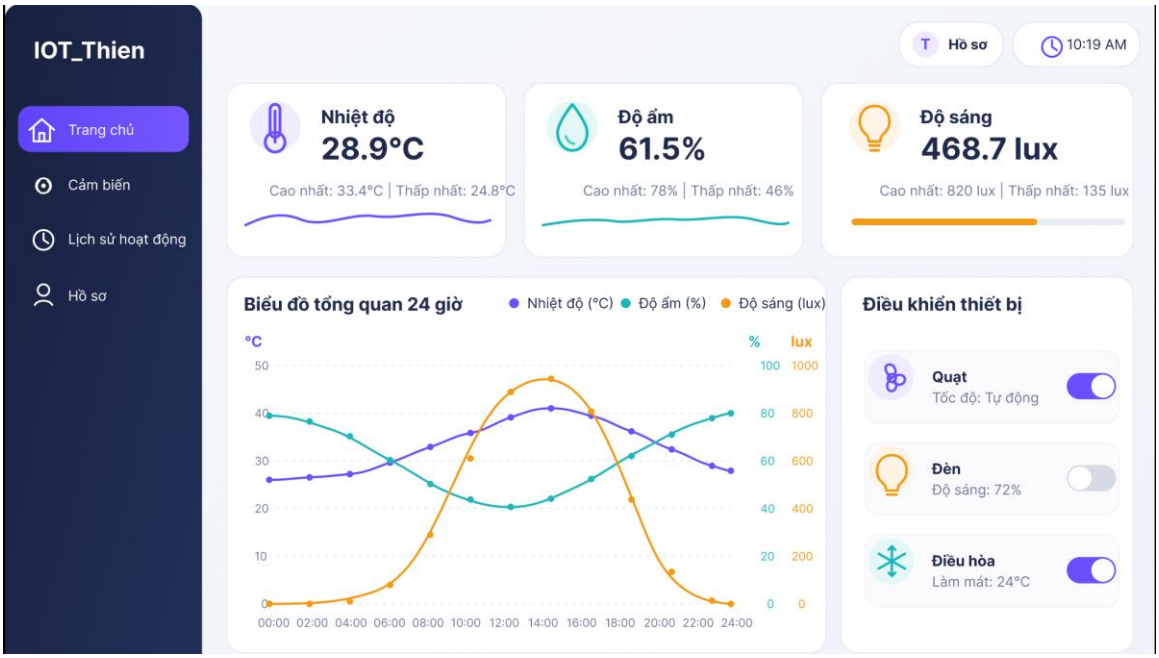
Chương 3 mô tả sáu use case theo cùng một cấu trúc. Ảnh giao diện được đặt trực tiếp ngay sau tiêu đề use case, không tách thành một mục độc lập; sau đó là đặc tả, sequence diagram, REST API, dữ liệu đọc/ghi tại CSDL và mapping API - Database. Riêng UC01 sử dụng ảnh Dashboard để thể hiện màn hình đích sau khi đăng nhập thành công. User là actor chính; ESP8266 là actor hệ thống tham gia ở luồng dữ liệu thời gian thực và điều khiển thiết bị.

Mã	Use case	Actor chính	Kết quả
UC01	Đăng nhập	User	Nhận JWT và truy cập Dashboard.
UC02	Xem dữ liệu cảm biến thời gian thực	User/ESP8266	Hiển thị giá trị mới nhất.
UC03	Xem biểu đồ dữ liệu cảm biến	User	Hiển thị chuỗi dữ liệu theo thời gian.
UC04	Điều khiển bật/tắt thiết bị	User/ESP8266	Thiết bị đổi trạng thái và lưu lịch sử.
UC05	Xem lịch sử giá trị cảm biến	User	Bảng sensor_data được lọc/phân trang.
UC06	Xem lịch sử điều khiển thiết bị	User	Bảng action_history được hiển thị.

Quy tắc tách sequence: mỗi hành động người dùng có một sequence riêng. Xem dữ liệu cảm biến thời gian thực, xem lịch sử giá trị cảm biến và xem lịch sử hoạt động thiết bị không gộp chung; mỗi sequence chỉ mô tả một mục tiêu nghiệp vụ từ giao diện đến Backend và CSDL.

3.1. Use case: Dashboard

Giao diện Dashboard sau khi đăng nhập thành công:



Hình 3.1. Giao diện Dashboard sau khi đăng nhập thành công

a. Đặc tả use case

Thuộc tính	Nội dung
Mã use case	UC01
Tên use case	Dashboard
Actor chính	User
Mục tiêu	Xác thực danh tính, cấp JWT và chuyển người dùng tới Dashboard.
Tiền điều kiện	Tài khoản tồn tại, chưa bị vô hiệu hóa; Frontend kết nối được Backend.
Hậu điều kiện	JWT và thông tin tối thiểu của User được lưu ở Frontend; người dùng vào Dashboard.
Sự kiện kích hoạt	User nhập username, password và chọn Đăng nhập.

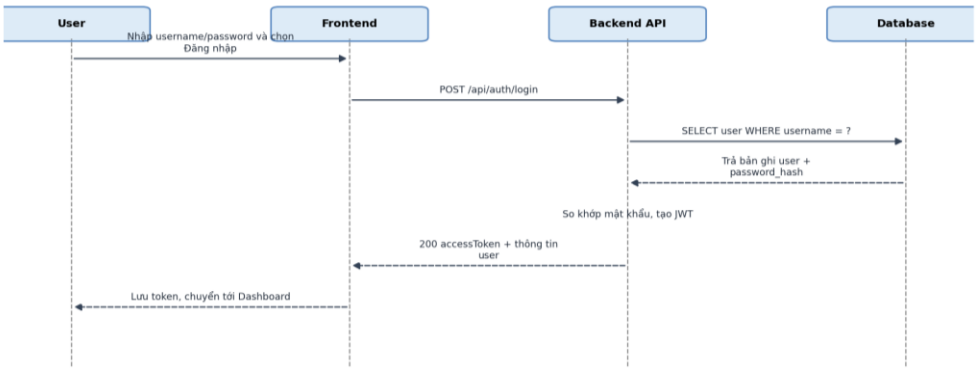
Luồng chính

1. Frontend kiểm tra hai trường bắt buộc và gửi POST /api/auth/login.
2. Backend tìm tài khoản theo username và so khớp password với password_hash.
3. Backend tạo JWT có thời hạn, trả token và thông tin người dùng không nhạy cảm.
4. Frontend lưu token theo cơ chế đã chọn và chuyển hướng tới Dashboard.

Luồng thay thế/ngoại lệ

- Sai thông tin: trả 401 với thông báo chung, không cho biết username hay password sai.
- Thiếu trường hoặc sai định dạng: trả 400 kèm danh sách lỗi validation.
- Lỗi hệ thống/CSDL: trả 500 và ghi log kỹ thuật, không trả stack trace cho client.

b. Sequence diagram



Hình 3.2. Sequence diagram đăng nhập

Mô tả các bước của sequence diagram:

Bước 1: User nhập tên đăng nhập (username), mật khẩu (password) và nhấn nút **Đăng nhập** trên giao diện.

Bước 2: Frontend kiểm tra các trường bắt buộc. Nếu username hoặc password bị bỏ trống, Frontend hiển thị lỗi tương ứng và không gửi yêu cầu tới Backend.

Bước 3: Khi dữ liệu hợp lệ, Frontend gửi yêu cầu `POST /api/auth/login` tới Backend API. Request gồm hai trường `username` và `password`.

Bước 4: Backend kiểm tra dữ liệu đầu vào, chuẩn hóa username và thực hiện truy vấn bảng `users` theo điều kiện `username = ?`.

Bước 5: Database trả về các trường cần thiết gồm `user_id`, `username`, `password_hash`, `full_name` và `role`. Nếu không tìm thấy tài khoản, Database trả về kết quả rỗng.

Bước 6: Backend sử dụng `bcrypt.compare()` để so sánh mật khẩu User vừa nhập với giá trị `password_hash` lưu trong bảng `users`.

Bước 7 – Đăng nhập thành công: Nếu username tồn tại và mật khẩu chính xác, Backend tạo JWT chứa các thông tin cần thiết như `userId`, `role` và thời hạn sử dụng `exp`.

Bước 8: Backend trả phản hồi HTTP `200` cho Frontend, gồm `accessToken`, `expiresIn` và thông tin User không nhạy cảm. Backend không trả `password_hash` về phía Frontend.

Bước 9: Frontend lưu token theo cơ chế của hệ thống, cập nhật trạng thái đăng nhập và chuyển User đến giao diện Dashboard.

Bước 10 – Sai thông tin đăng nhập: Nếu username không tồn tại hoặc mật khẩu không chính xác, Backend trả phản hồi HTTP `401` với thông báo chung: "Tên đăng nhập hoặc mật khẩu không đúng".

Bước 11: Frontend hiển thị thông báo lỗi và giữ User tại trang đăng nhập. Hệ thống không cho biết cụ thể username hay mật khẩu bị sai nhằm hạn chế lộ thông tin tài khoản.

c. Mô tả API

Method	Endpoint	Xác thực	Mô tả/Kết quả
POST	/api/auth/login	Không	Xác thực tài khoản; 200 trả accessToken và user.
GET	/api/auth/me	Bearer JWT	Lấy thông tin phiên hiện tại; 200 trả user.

```
Request
{"username":"phamminhthien","password":"********"}

Response 200
{"accessToken":"eyJ...","expiresIn":3600,"user":{"userId":1,"fullName":"Phạm Minh
Thiện","role":"USER"}}
```

d. Dữ liệu đọc/ghi tại CSDL

Nghiệp vụ	Bảng	Trường đọc/ghi	Thao tác/Giá trị
Đọc tài khoản	users	username, user_id, password_hash, full_name, role	SELECT theo username; không lưu mật khẩu rõ
Đọc hồ sơ	users	user_id, full_name, student_id, email, role	SELECT theo user_id lấy từ JWT
Ghi dữ liệu	Không phát sinh	-	Đăng nhập chỉ đọc; JWT được tạo tại Backend

e. Mapping dữ liệu API - Database

Dữ liệu API	Kiểm tra/xử lý	Bảng và thao tác CSDL	Dữ liệu trả về
username	trim, bắt buộc, tối đa 50 ký tự	SELECT users WHERE username = ?; password_hash chỉ dùng nội bộ	user_id, role
password	bcrypt.compare; không ghi log	Không lưu/không truy vấn trực tiếp ngoài hash	Không trả về
JWT claims	userId, role, exp	Không bắt buộc lưu session	accessToken, expiresIn
GET /me	Đọc userId từ JWT	SELECT trường hồ sơ FROM users	Thông tin User an toàn

3.2. Use case: Xem dữ liệu cảm biến thời gian thực

Giao diện sử dụng trong use case:



Hình 3.3. Khu vực hiển thị dữ liệu cảm biến mới nhất trên Dashboard

a. Đặc tả use case

Thuộc tính	Nội dung
Mã use case	UC02
Tên use case	Xem dữ liệu cảm biến thời gian thực
Actor chính	User; ESP8266 (actor hệ thống)
Mục tiêu	Quan sát giá trị nhiệt độ, độ ẩm và ánh sáng mới nhất.
Tiền điều kiện	User đã đăng nhập; Backend, Broker và ESP8266 hoạt động hoặc CSDL có bản ghi gần nhất.
Hậu điều kiện	Dashboard hiển thị giá trị, thời điểm đo, trạng thái NORMAL/WARNING và độ mới dữ liệu.
Sự kiện kích hoạt	User mở Dashboard; Frontend tự làm mới theo chu kỳ.

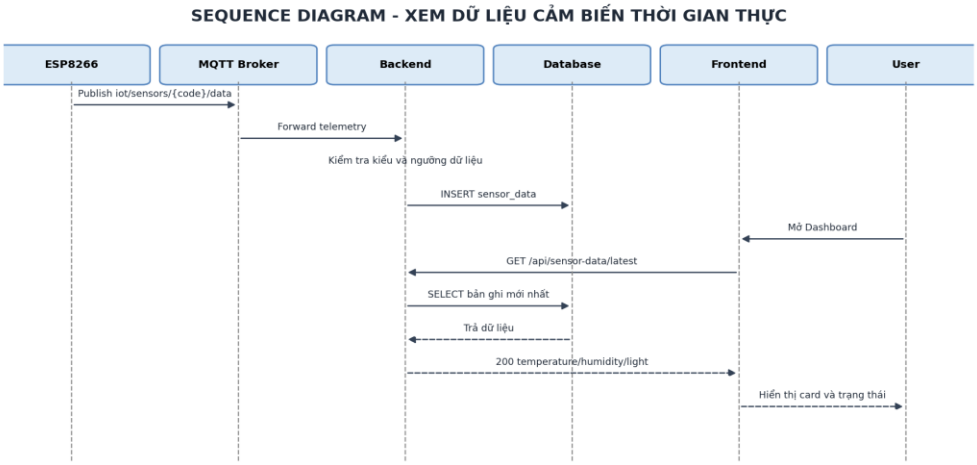
Luồng chính

1. ESP8266 publish telemetry; Backend kiểm tra và ghi sensor_data.
2. Frontend gọi GET /api/sensor-data/latest với JWT.
3. Backend lấy bản ghi mới nhất của cảm biến và tính dataAge.
4. Frontend cập nhật các card; nếu vượt ngưỡng thì hiển thị trạng thái cảnh báo.

Luồng thay thế/ngoại lệ

- Không có dữ liệu: trả 404 hoặc payload data = null để Frontend hiển thị Chưa có dữ liệu.
- Dữ liệu quá cũ: API vẫn trả bản ghi gần nhất nhưng đánh dấu stale = true.
- JWT hết hạn: trả 401; Frontend chuyển về trang đăng nhập.

b. Sequence diagram



Hình 3.4. Sequence diagram xem dữ liệu cảm biến thời gian thực

Mô tả các bước của sequence diagram:

Bước 1: ESP8266 đọc dữ liệu từ các cảm biến theo chu kỳ, bao gồm nhiệt độ, độ ẩm và cường độ ánh sáng.

Bước 2: ESP8266 tạo payload JSON chứa `sensorCode`, `temperature`, `humidity`, `light` và `timestamp`.

Bước 3: ESP8266 publish payload lên MQTT Broker thông qua topic `iot/sensors/{sensorCode}/data`.

Bước 4: MQTT Broker tiếp nhận message và chuyển tiếp dữ liệu tới Backend đang subscribe topic telemetry.

Bước 5: Backend kiểm tra cấu trúc payload, kiểu dữ liệu, mã cảm biến và phạm vi hợp lệ của các giá trị đo.

Bước 6: Backend truy vấn bảng `sensors` theo `sensor_code` để xác định `sensor_id` và tên cảm biến.

Bước 7: Backend lưu dữ liệu vào bảng `sensor_data`. Các trường được ghi gồm `sensor_id`, `temperature`, `humidity`, `light`, `status` và `recorded_at`. Trường `data_id` được Database tự động tăng.

Bước 8: Khi User mở Dashboard, Frontend gửi yêu cầu `GET /api/sensor-data/latest` kèm JWT tới Backend.

Bước 9: Backend kiểm tra JWT, sau đó truy vấn bản ghi mới nhất trong bảng `sensor_data` theo `sensor_id`, sắp xếp `recorded_at` giảm dần và lấy một bản ghi đầu tiên.

Bước 10: Database trả dữ liệu cảm biến mới nhất cho Backend. Backend tính thêm độ trễ dữ liệu `dataAgeSeconds` và xác định dữ liệu có bị cũ hay không.

Bước 11: Backend trả phản hồi HTTP `200`, gồm tên cảm biến, nhiệt độ, độ ẩm, ánh sáng, trạng thái, thời gian đo và độ mới của dữ liệu.

Bước 12: Frontend hiển thị các giá trị lên các thẻ thông tin trên Dashboard. Nếu giá trị vượt ngưỡng, Frontend hiển thị trạng thái cảnh báo.

Bước 13: Frontend tự động gọi lại API theo chu kỳ, ví dụ 5 giây một lần, để cập nhật dữ liệu gần thời gian thực.

c. Mô tả API

Method	Endpoint	Xác thực	Mô tả/Kết quả
GET	/api/sensor-data/latest?sensorCode=SENSOR_01	Bearer JWT	Trả bản ghi mới nhất và dataAge.
GET	/api/sensors	Bearer JWT	Trả danh sách cảm biến đang hoạt động.

```
Response 200
{"sensorCode":"SENSOR_01","temperature":28.9,"humidity":61.5,"light":468.7,
 "status":"NORMAL","recordedAt":"2026-08-
24T13:20:15Z","stale":false,"dataAgeSeconds":3}
```

d. Dữ liệu đọc/ghi tại CSDL

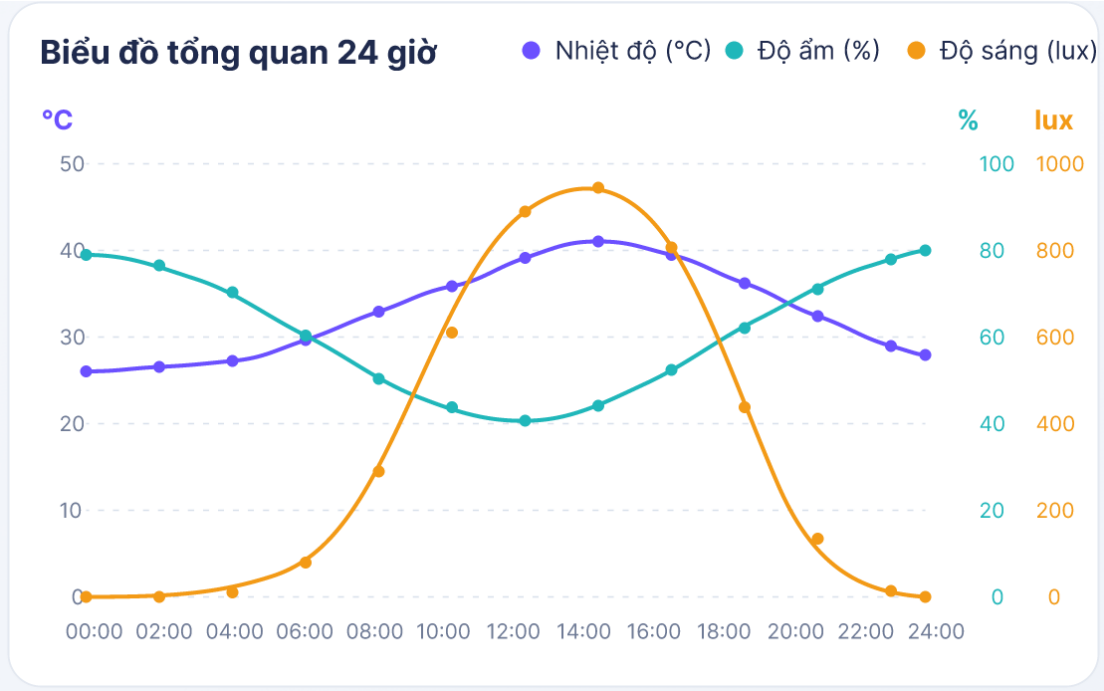
Nghiệp vụ	Bảng	Trường đọc/ghi	Thao tác/Giá trị
Đối chiếu cảm biến	sensors	sensor_id, sensor_code, sensor_name	SELECT sensor_id theo sensor_code
Lưu telemetry	sensor_data	sensor_id, temperature, humidity, light, status, recorded_at	INSERT; data_id được AUTO_INCREMENT
Đọc mới nhất	sensor_data	data_id, temperature, humidity, light, status, recorded_at	SELECT ORDER BY recorded_at DESC LIMIT 1

e. Mapping dữ liệu API - Database

Dữ liệu API	Kiểm tra/xử lý	Bảng và thao tác CSDL	Dữ liệu trả về
sensorCode	Tùy chọn; phải tồn tại	sensors.sensor_code → sensor_id	sensorCode, sensorName
latest	ORDER BY recorded_at DESC LIMIT 1	SELECT sensor_data WHERE sensor_id = ?	temperature, humidity, light
status	Đọc trạng thái đã lưu	sensor_data.status	NORMAL/WARNING
dataAge	now - recorded_at	Không ghi CSDL	stale, dataAgeSeconds

3.3. Use case: Xem biểu đồ dữ liệu cảm biến

Giao diện sử dụng trong use case:



Hình 3.5. Biểu đồ kết hợp nhiệt độ, độ ẩm và ánh sáng

a. Đặc tả use case

Thuộc tính	Nội dung
Mã use case	UC03
Tên use case	Xem biểu đồ dữ liệu cảm biến
Actor chính	User
Mục tiêu	Quan sát xu hướng của ba đại lượng trong khoảng thời gian lựa chọn.
Tiền điều kiện	User đã đăng nhập; sensor_data có dữ liệu trong khoảng thời gian.
Hậu điều kiện	Biểu đồ hiển thị labels và ba series đồng bộ theo cùng mốc thời gian.
Sự kiện kích hoạt	User mở Dashboard hoặc thay đổi bộ lọc thời gian.

Luồng chính

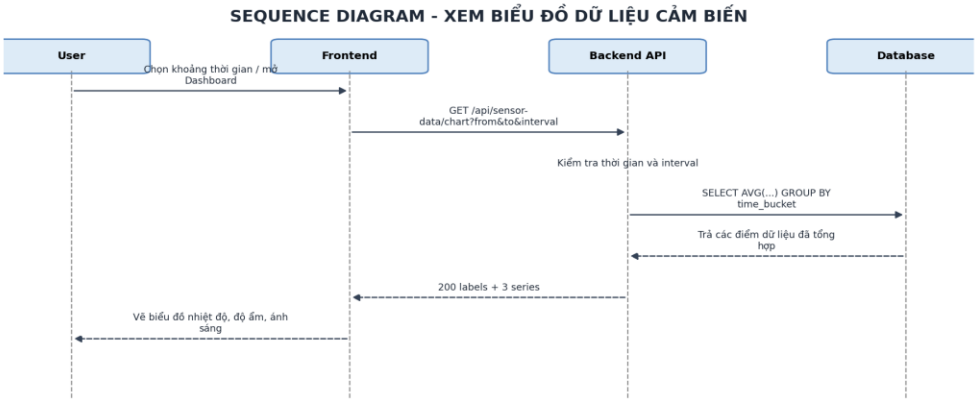
1. Frontend gửi from, to, interval và sensorCode tới API chart.
2. Backend kiểm tra khoảng thời gian, chuẩn hóa múi giờ và giới hạn số điểm.
3. CSDL tổng hợp AVG theo time bucket; Backend lấp các khoảng trống bằng null.
4. Frontend vẽ ba đường dữ liệu và tooltip theo thời gian.

Luồng thay thế/ngoại lệ

- from ≥ to hoặc interval không hợp lệ: trả 400.
- Khoảng dữ liệu rỗng: trả labels/series rỗng để hiển thị thông báo.

- Khoảng thời gian quá lớn: Backend tự tăng interval hoặc trả 422.

b. Sequence diagram



Hình 3.6. Sequence diagram xem biểu đồ dữ liệu cảm biến

Mô tả các bước của sequence diagram:

Bước 1: User mở Dashboard hoặc lựa chọn khoảng thời gian cần xem biểu đồ.

Bước 2: Frontend thu thập các tham số gồm `from`, `to`, `interval` và `sensorCode`.

Bước 3: Frontend gửi yêu cầu `GET /api/sensor-data/chart` kèm các tham số và JWT tới Backend API.

Bước 4: Backend kiểm tra JWT, kiểm tra định dạng thời gian, bảo đảm `from < to` và kiểm tra `interval` có thuộc danh sách cho phép hay không.

Bước 5: Backend chuẩn hóa múi giờ và giới hạn khoảng thời gian truy vấn để tránh trả về quá nhiều điểm dữ liệu.

Bước 6: Backend truy vấn bảng `sensors` để chuyển `sensorCode` thành `sensor_id`.

Bước 7: Backend gửi câu truy vấn tới Database để lấy dữ liệu trong bảng `sensor_data` theo `sensor_id` và khoảng thời gian đã chọn.

Bước 8: Database tổng hợp giá trị trung bình của `temperature`, `humidity` và `light` theo từng nhóm thời gian (time bucket) bằng phép tính `AVG` và `GROUP BY`.

Bước 9: Database trả danh sách các điểm dữ liệu đã tổng hợp cho Backend.

Bước 10: Backend sắp xếp dữ liệu theo thời gian tăng dần, làm tròn giá trị và bổ sung `null` tại các mốc không có dữ liệu để các chuỗi có cùng số phần tử.

Bước 11: Backend trả phản hồi HTTP `200`, gồm `labels`, chuỗi nhiệt độ, chuỗi độ ẩm, chuỗi ánh sáng, `interval` và `timezone`.

Bước 12: Frontend nhận dữ liệu và vẽ biểu đồ kết hợp ba đường nhiệt độ, độ ẩm và ánh sáng. Khi User di chuột vào một mốc thời gian, Frontend hiển thị tooltip chứa các giá trị tương ứng.

Method	Endpoint	Xác thực	Mô tả/Kết quả
GET	/api/sensor-data/chart?from&to&interval&sensorCode	Bearer JWT	Trả labels và các series đã tổng hợp.

```
Response 200
{"labels":["13:00","14:00","15:00"],

"series":{"temperature":[28.5,28.9,29.1],"humidity":[62.0,61.5,60.8],"light":[420,468,5
10]},
 "interval":"1h","timezone":"Asia/Bangkok"}
```

d. Dữ liệu đọc/ghi tại CSDL

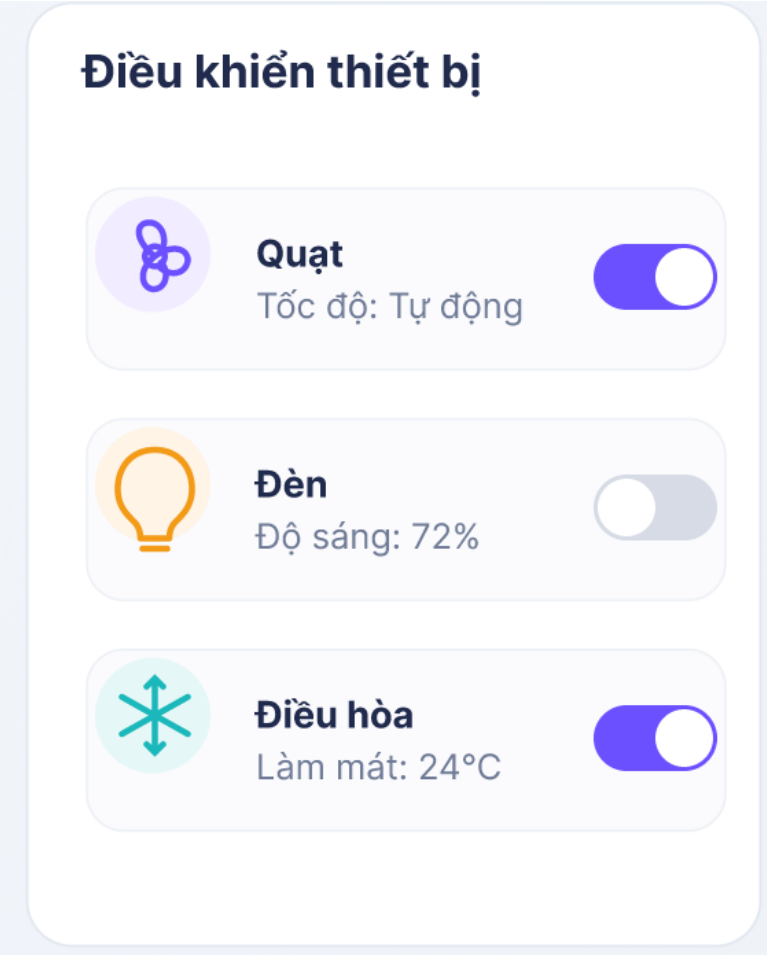
Nghiệp vụ	Bảng	Trường đọc/ghi	Thao tác/Giá trị
Xác định cảm biến	sensors	sensor_id, sensor_code, sensor_name	SELECT/JOIN theo sensor_code
Tạo chuỗi biểu đồ	sensor_data	sensor_id, temperature, humidity, light, recorded_at	SELECT + AVG/GROUP BY time bucket
Ghi dữ liệu	Không phát sinh	-	Use case chỉ đọc, không INSERT/UPDATE

e. Mapping dữ liệu API - Database

Dữ liệu API	Kiểm tra/xử lý	Bảng và thao tác CSDL	Dữ liệu trả về
from, to	ISO-8601; from < to; giới hạn phạm vi	WHERE recorded_at BETWEEN ? AND ?	Khoảng thời gian thực tế
interval	Whitelist 1m/5m/1h/1d	GROUP BY time bucket	interval
sensorCode	Đổi sang sensor_id	JOIN sensors; lọc sensor_data.sensor_id	Tên/mã cảm biến
series	AVG + ROUND; sắp tăng theo thời gian	AVG temperature/humidity/light	labels + 3 mảng số

3.4. Use case: Điều khiển bật/tắt thiết bị

Giao diện sử dụng trong use case:



Hình 3.7. Khu vực điều khiển thiết bị trên Dashboard

a. Đặc tả use case

Thuộc tính	Nội dung
Mã use case	UC04
Tên use case	Điều khiển bật/tắt thiết bị
Actor chính	User; ESP8266 (actor hệ thống)
Mục tiêu	Thay đổi trạng thái thiết bị và ghi nhận kết quả có thể kiểm tra.
Tiền điều kiện	User đăng nhập; thiết bị tồn tại; Backend kết nối Broker; ESP8266 online.
Hậu điều kiện	devices.status phản ánh ACK; action_history là SUCCESS hoặc FAILED.
Sự kiện kích hoạt	User chọn ON/OFF trên công tắc của một thiết bị.

Luồng chính

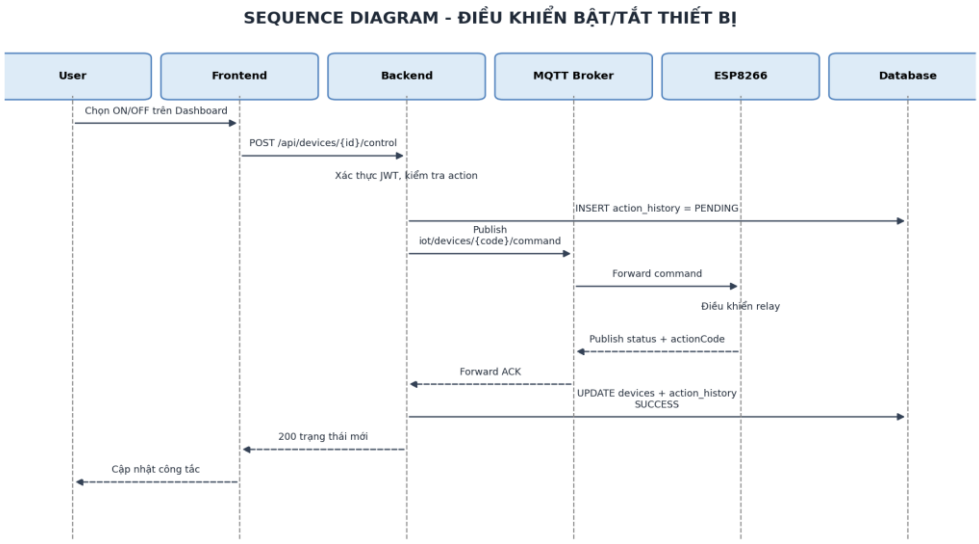
1. Frontend gửi deviceId, action và mode tới API control.

2. Backend xác thực, kiểm tra thiết bị, tạo actionCode và bản ghi PENDING.
3. Backend publish command; ESP8266 điều khiển relay và publish status.
4. Backend đối chiếu actionCode, cập nhật device và history rồi trả kết quả.
5. Frontend cập nhật công tắc theo trạng thái xác nhận, không theo trạng thái dự đoán.

Luồng thay thế/ngoại lệ

- Thiết bị offline: trả 409/503 và ghi FAILED, không cập nhật devices.status.
- Không nhận ACK: hết timeout, action_history chuyển FAILED.
- Action không thuộc ON/OFF/AUTO: trả 400 trước khi publish MQTT.

b. Sequence diagram



Hình 3.8. Sequence diagram điều khiển bật/tắt thiết bị

Mô tả các bước của sequence diagram:

Bước 1: User chọn trạng thái **ON** hoặc **OFF** tại công tắc điều khiển thiết bị trên Dashboard.

Bước 2: Frontend lấy `deviceId`, trạng thái điều khiển `action` và chế độ `mode`, sau đó gửi yêu cầu `POST /api/devices/{deviceId}/control` tới Backend.

Bước 3: Backend kiểm tra JWT, quyền điều khiển của User và tính hợp lệ của `deviceId`, `action` và `mode`.

Bước 4: Backend truy vấn bảng `devices` để kiểm tra thiết bị có tồn tại hay không, đồng thời lấy `device_code`, trạng thái hiện tại và thời điểm hoạt động gần nhất.

Bước 5: Backend tạo một `actionCode` duy nhất để theo dõi lệnh điều khiển.

Bước 6: Backend thêm một bản ghi vào bảng `action_history`. Các trường được ghi gồm `action_code`, `device_id`, `user_id`, `action`, `requested_state`, `result`, `area` và `action_time`. Ví dụ khi User bật thiết bị, hệ thống lưu `action = 'ON'`, `requested_state = 'ON'` và `result = 'PENDING'`.

Bước 7: Backend publish lệnh điều khiển lên MQTT Broker thông qua topic `iot/devices/{deviceCode}/command`. Payload gồm `actionCode`, `deviceCode`, `action`, `mode` và `requestedAt`.

Bước 8: MQTT Broker chuyển tiếp lệnh tới ESP8266 đang subscribe topic command.

Bước 9: ESP8266 kiểm tra lệnh, điều khiển chân GPIO và relay để bật hoặc tắt thiết bị vật lý.

Bước 10: Sau khi điều khiển, ESP8266 publish message xác nhận (ACK) lên topic trạng thái. Message chứa `actionCode`, `deviceCode`, trạng thái thực tế và thời gian xác nhận.

Bước 11: MQTT Broker chuyển message ACK tới Backend.

Bước 12: Backend đối chiếu `actionCode` và `deviceCode` để bảo đảm ACK thuộc đúng lệnh điều khiển.

Bước 13: Nếu ACK hợp lệ, Backend cập nhật bảng `devices`, gồm các trường `status`, `mode`, `last_seen` và `updated_at`.

Bước 14: Backend cập nhật bản ghi tương ứng trong bảng `action_history`, chuyển `result` từ `PENDING` thành `SUCCESS`.

Bước 15: Backend trả HTTP `200` cho Frontend, gồm `actionCode`, `deviceId`, trạng thái mới, chế độ, kết quả và thời gian xác nhận.

Bước 16: Frontend cập nhật trạng thái công tắc theo trạng thái đã được ESP8266 xác nhận. Frontend không cập nhật vĩnh viễn theo trạng thái dự đoán trước khi nhận ACK.

c. Mô tả API

Method	Endpoint	Xác thực	Mô tả/Kết quả
POST	/api/devices/{deviceId}/control	Bearer JWT	Tạo lệnh; 200 khi nhận ACK, 202 nếu xử lý bất đồng bộ.
GET	/api/devices	Bearer JWT	Lấy trạng thái và lastSeen của thiết bị.

```
Request
{"action":"ON","mode":"MANUAL"}

Response 200
{"actionCode":"9a8b...","deviceId":2,"status":"ON","mode":"MANUAL","result":"SUCCESS","
confirmedAt":"2026-08-24T13:21:02Z"}
```

d. Dữ liệu đọc/ghi tại CSDL

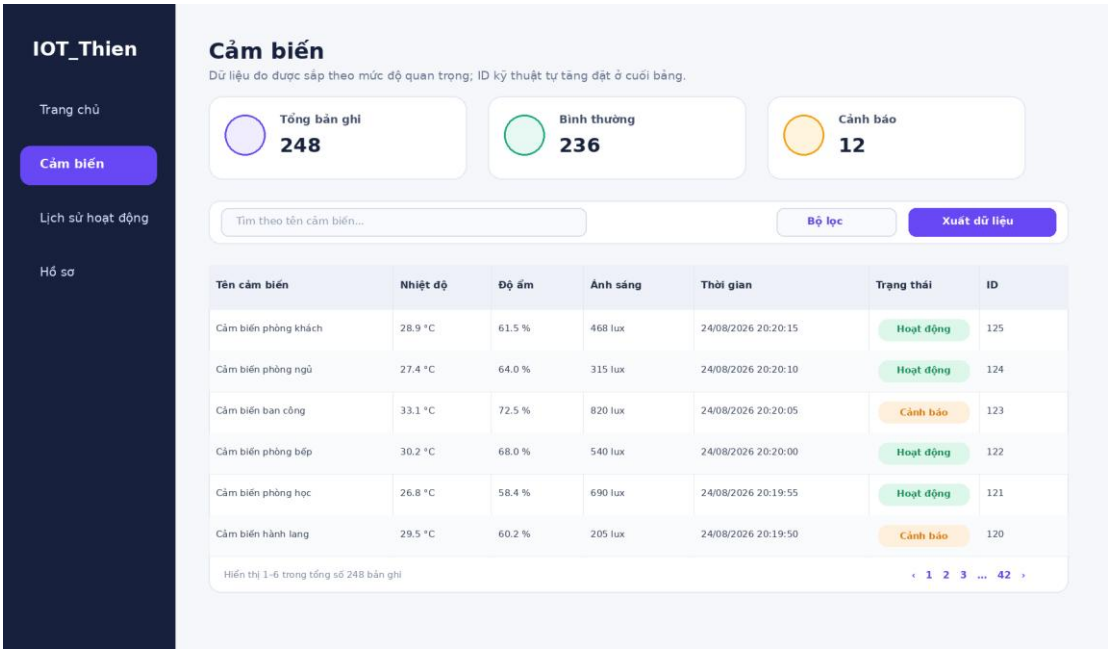
Nghiệp vụ	Bảng	Trường đọc/ghi	Thao tác/Giá trị
Tạo lệnh	action_history	action_code, device_id, user_id, action, requested_state, result, area, action_time	INSERT; ví dụ action='ON', requested_state='ON', result='PENDING'
ACK thành công	action_history	result	UPDATE result='SUCCESS'
Timeout/lỗi	action_history	result	UPDATE result='FAILED'
Đồng bộ thiết bị	devices	status, mode, last_seen, updated_at	UPDATE sau ACK; ví dụ status='ON'

e. Mapping dữ liệu API - Database

Dữ liệu API	Kiểm tra/xử lý	Bảng và thao tác CSDL	Dữ liệu trả về
deviceId	Số nguyên; thiết bị tồn tại	SELECT devices; lấy device_code	deviceId, deviceCode
action/mode	Whitelist ON/OFF/AUTO; MANUAL/AUTO	INSERT action_history = PENDING	actionCode, result
MQTT ACK	Đúng actionCode và deviceCode	UPDATE devices.status/mode/last_seen	status, confirmedAt
Kết quả	ACK → SUCCESS; timeout → FAILED	UPDATE action_history.result	result, error nếu có

3.5. Use case: Xem lịch sử giá trị cảm biến

Giao diện sử dụng trong use case:



Hình 3.9. Giao diện Data Sensor

a. Đặc tả use case

Thuộc tính	Nội dung
Mã use case	UC05
Tên use case	Xem lịch sử giá trị cảm biến
Actor chính	User
Mục tiêu	Tra cứu các lần đo theo cảm biến, thời gian, trạng thái và thứ tự.
Tiền điều kiện	User đã đăng nhập; hệ thống có quyền đọc sensor_data.
Hậu điều kiện	Bảng hiển thị đúng bộ lọc, tổng số bản ghi, số trang và thứ tự.
Sự kiện kích hoạt	User mở Data Sensor hoặc thay đổi tìm kiếm/bộ lọc/phân trang.

Luồng chính

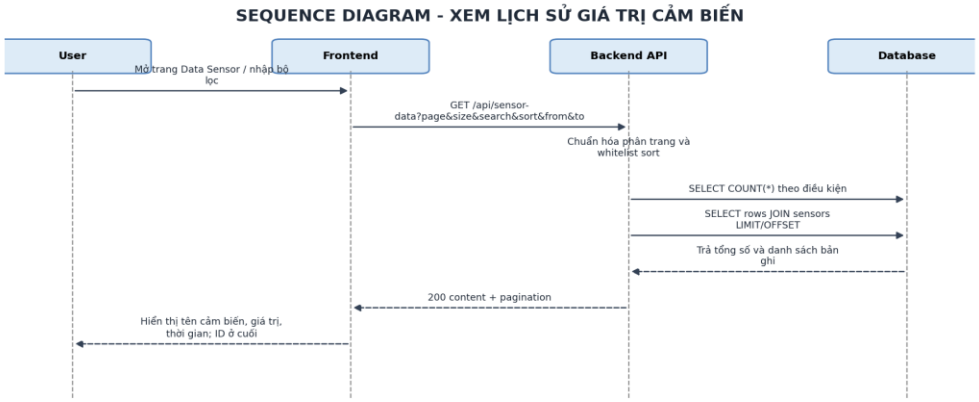
1. Frontend gửi page, size, search, sort, from, to và status.
2. Backend chuẩn hóa phân trang, whitelist cột sort và dựng điều kiện tham số hóa.
3. CSDL đếm tổng số rồi lấy trang dữ liệu JOIN sensors.
4. Frontend hiển thị theo thứ tự ưu tiên: tên cảm biến, các giá trị đo, thời gian, trạng thái và data_id ở cuối.

Luồng thay thế/ngoại lệ

- page/size không hợp lệ: dùng giá trị mặc định hoặc trả 400 theo quy ước API.
- Không có bản ghi phù hợp: trả content rỗng, totalElements = 0.

- Tên cột sort không được cho phép: trả 400, không ghép trực tiếp vào SQL.

b. Sequence diagram



Hình 3.10. Sequence diagram xem lịch sử giá trị cảm biến

Mô tả các bước của sequence diagram:

Bước 1: User mở giao diện **Data Sensor** hoặc thay đổi nội dung tìm kiếm, bộ lọc thời gian, trạng thái, cách sắp xếp hoặc trang dữ liệu.

Bước 2: Frontend gửi yêu cầu `GET /api/sensor-data` tới Backend. Các tham số có thể gồm `page`, `size`, `search`, `sort`, `from`, `to` và `status`.

Bước 3: Backend kiểm tra JWT và quyền truy cập dữ liệu cảm biến của User.

Bước 4: Backend chuẩn hóa các tham số phân trang, bảo đảm `page >= 0` và giới hạn `size` trong phạm vi cho phép.

Bước 5: Backend kiểm tra trường sắp xếp theo whitelist. Hệ thống không ghép trực tiếp tên cột do User gửi vào câu SQL.

Bước 6: Backend chuẩn hóa nội dung tìm kiếm, kiểm tra khoảng thời gian và kiểm tra giá trị `status`.

Bước 7: Backend yêu cầu Database thực hiện `COUNT(*)` để xác định tổng số bản ghi phù hợp với điều kiện lọc.

Bước 8: Backend tiếp tục truy vấn `sensor_data JOIN sensors` để lấy một trang dữ liệu bằng `LIMIT` và `OFFSET`.

Bước 9: Database trả tổng số bản ghi và danh sách dữ liệu gồm `sensor_name`, `temperature`, `humidity`, `light`, `recorded_at`, `status` và `data_id`.

Bước 10: Backend chuyển đổi tên trường Database sang tên trường API, tính `totalPages` và tạo metadata phân trang.

Bước 11: Backend trả phản hồi HTTP `200`, gồm `content`, `page`, `size`, `totalElements`, `totalPages` và thông tin sắp xếp.

Bước 12: Frontend hiển thị dữ liệu theo mức độ quan trọng: tên cảm biến, nhiệt độ, độ ẩm, ánh sáng, thời gian và trạng thái được đặt trước; trường kỹ thuật `data_id` tự tăng được đặt ở cuối bảng.

Bước 13: Khi User chuyển trang hoặc thay đổi bộ lọc, Frontend gửi lại yêu cầu với tham số mới và cập nhật bảng dữ liệu.

c. Mô tả API

Method	Endpoint	Xác thực	Mô tả/Kết quả
GET	/api/sensor-data?page&size&search&sort&from&to&status	Bearer JWT	Danh sách lịch sử cảm biến có phân trang.
GET	/api/sensor-data/{dataId}	Bearer JWT	Chi tiết một bản ghi.

```
Response 200
{"content":[{"sensorName":"Cảm biến phòng khách","temperature":28.9,"humidity":61.5,
 "light":468,"recordedAt":"2026-08-24T13:20:15Z","status":"NORMAL","dataId":125}],
 "page":0,"size":10,"totalElements":248,"totalPages":25,"sort":"recordedAt,desc"}
```

d. Dữ liệu đọc/ghi tại CSDL

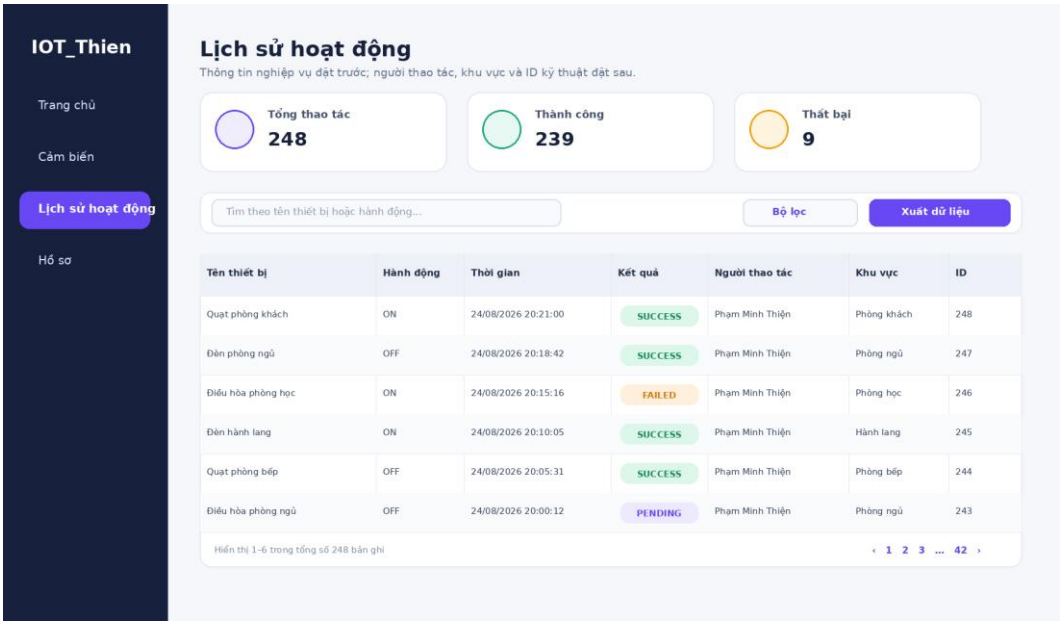
Nghiệp vụ	Bảng	Trường đọc/ghi	Thao tác/Giá trị
Hiển thị lịch sử	sensor_data JOIN sensors	sensor_name, temperature, humidity, light, recorded_at, status, data_id	SELECT/JOIN; data_id là AUTO_INCREMENT
Lọc/phân trang	sensor_data	recorded_at, status, data_id	WHERE + ORDER BY + LIMIT/OFFSET
Ghi dữ liệu	Không phát sinh	-	Use case chỉ đọc; dữ liệu đã được ghi từ telemetry

e. Mapping dữ liệu API - Database

Dữ liệu API	Kiểm tra/xử lý	Bảng và thao tác CSDL	Dữ liệu trả về
page, size	page ≥ 0; 1 ≤ size ≤ 100	LIMIT ? OFFSET ?	page, size, totalPages
search	trim; escape ký tự LIKE	sensor_code/name/location LIKE ?	Danh sách phù hợp
from, to, status	ISO-8601; enum hợp lệ	WHERE recorded_at/status	Bộ lọc đã áp dụng
sort	Map API field → cột whitelist	ORDER BY recorded_at/temperature/...	sort
content	JOIN để lấy tên cảm biến; ID kỹ thuật đặt cuối	sensor_data JOIN sensors	sensorName, temperature, humidity, light, recordedAt, status, dataId

3.6. Use case: Xem lịch sử điều khiển thiết bị

Giao diện sử dụng trong use case:



Hình 3.11. Giao diện Action History

a. Đặc tả use case

Thuộc tính	Nội dung
Mã use case	UC06
Tên use case	Xem lịch sử điều khiển thiết bị
Actor chính	User
Mục tiêu	Kiểm tra ai đã điều khiển thiết bị nào, hành động gì, thời gian và kết quả.
Tiền điều kiện	User đã đăng nhập; action_history có thể được truy vấn.
Hậu điều kiện	Bảng lịch sử hiển thị đúng bản ghi và không làm thay đổi nhật ký.
Sự kiện kích hoạt	User mở Action History hoặc thay đổi khoảng thời gian/bộ lọc.

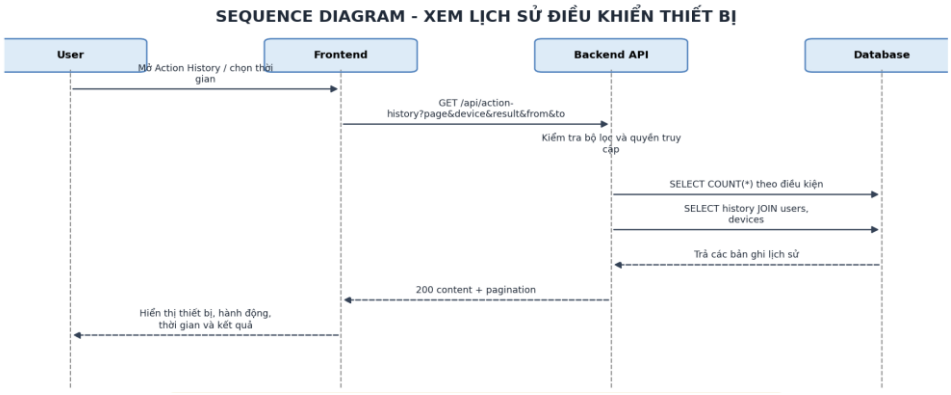
Luồng chính

1. Frontend gửi page, size, device, action, result, from và to.
2. Backend xác thực quyền đọc, kiểm tra bộ lọc và dựng truy vấn tham số hóa.
3. CSDL JOIN action_history với users và devices, đồng thời đếm tổng số.
4. Frontend hiển thị tên thiết bị, action, thời gian và kết quả trước; người thao tác, khu vực và history_id đặt sau.

Luồng thay thế/ngoại lệ

- Khoảng thời gian không hợp lệ: trả 400.
- Không có dữ liệu: trả content rỗng thay vì coi là lỗi hệ thống.
- User không có quyền xem toàn bộ lịch sử: trả 403 hoặc tự giới hạn theo user_id.

b. Sequence diagram



Hình 3.12. Sequence diagram xem lịch sử điều khiển thiết bị

Mô tả các bước của sequence diagram:

Bước 1: User mở giao diện **Action History** hoặc lựa chọn bộ lọc theo thiết bị, hành động, kết quả và khoảng thời gian.

Bước 2: Frontend gửi yêu cầu `GET /api/action-history` tới Backend. Các tham số có thể gồm `page`, `size`, `deviceId`, `action`, `result`, `from` và `to`.

Bước 3: Backend kiểm tra JWT và quyền xem lịch sử điều khiển của User.

Bước 4: Backend kiểm tra `deviceId`, các giá trị enum `action`, `result`, khoảng thời gian và tham số phân trang.

Bước 5: Backend xây dựng điều kiện truy vấn tham số hóa để hạn chế nguy cơ SQL Injection.

Bước 6: Backend yêu cầu Database thực hiện `COUNT(*)` trên bảng `action_history` để xác định tổng số bản ghi phù hợp.

Bước 7: Backend truy vấn `action_history JOIN devices JOIN users` để lấy thông tin lịch sử, tên thiết bị và người thực hiện thao tác.

Bước 8: Database trả các trường gồm `device_name`, `action`, `action_time`, `result`, `full_name`, `area`, `history_id` và `action_code`.

Bước 9: Backend chuyển đổi dữ liệu sang cấu trúc API, tính tổng số trang và bổ sung metadata phân trang.

Bước 10: Backend trả phản hồi HTTP `200`, gồm danh sách lịch sử và các thông tin `page`, `size`, `totalElements` và `totalPages`.

Bước 11: Frontend hiển thị các trường quan trọng gồm tên thiết bị, hành động `ON/OFF`, thời gian và kết quả `PENDING/SUCCESS/FAILED` ở phía trước.

Bước 12: Các trường bổ sung như người thao tác, khu vực và `history_id` được đặt phía sau vì có mức độ ưu tiên thấp hơn đối với User.

Bước 13: Khi User thay đổi bộ lọc hoặc chuyển trang, Frontend gửi yêu cầu mới và cập nhật lại bảng Action History.

c. Mô tả API

Method	Endpoint	Xác thực	Mô tả/Kết quả
GET	/api/action-history?page&size&deviceId&action&result&from&to	Bearer JWT	Danh sách lịch sử điều khiển có phân trang.
GET	/api/action-history/{historyId}	Bearer JWT	Chi tiết một thao tác và actionCode.

```
Response 200
{"content":[{"deviceName":"Quạt","action":"ON","actionTime":"2026-08-24T13:21:00Z",
 "result":"SUCCESS","userName":"Phạm Minh Thiện","area":"Phòng
A","historyId":248,"actionCode":"9a8b..."}],
 "page":0,"size":10,"totalElements":248,"totalPages":25}
```

d. Dữ liệu đọc/ghi tại CSDL

Nghiệp vụ	Bảng	Trường đọc/ghi	Thao tác/Giá trị
Hiển thị lịch sử	action_history JOIN devices JOIN users	device_name, action, action_time, result, full_name, area, history_id	SELECT; trường phụ và ID kỹ thuật đặt cuối
Lọc/phân trang	action_history	device_id, action, result, action_time	WHERE + ORDER BY + LIMIT/OFFSET
Ghi dữ liệu	Không phát sinh	-	Use case chỉ đọc; bản ghi được tạo ở UC04

e. Mapping dữ liệu API - Database

Dữ liệu API	Kiểm tra/xử lý	Bảng và thao tác CSDL	Dữ liệu trả về
page, size	Giới hạn tối đa 100	LIMIT/OFFSET action_history	Metadata phân trang
deviceId/action/result	Kiểm tra ID và enum	WHERE device_id/action/result	Bộ lọc
from, to	ISO-8601; from < to	WHERE action_time BETWEEN ? AND ?	Khoảng thời gian
content	Ẩn dữ liệu nhạy cảm; sắp cột theo mức ưu tiên	JOIN devices, users	deviceName, action, actionTime, result, userName, area, historyId